

NAME: D.Teja Sadwika

RollNo:2520030536

OSSP-Practical

1.Virtual Addressing

```
sadwika@DESKTOP-8CQA01L x + v
GNU nano 8.7.1 void.c
#include <stdio.h>
int main() {
    int x = 10;
    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);
    return 0;
}
```

```
sadwika@DESKTOP-8CQA01L:~/Skill$ nano void.c
sadwika@DESKTOP-8CQA01L:~/Skill$ gcc void.c -o void
sadwika@DESKTOP-8CQA01L:~/Skill$ ./void
Value of x = 10
Virtual address of x = 0x7ffc70d3beb4
```

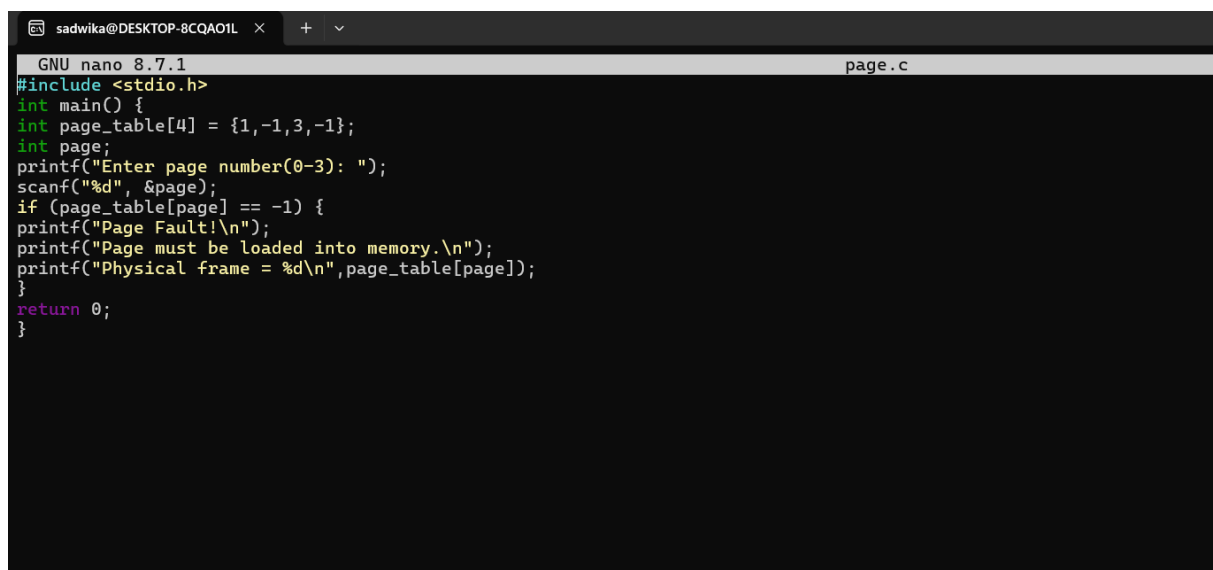
2. Address Space

```
sadwika@DESKTOP-8CQA01L x + v
GNU nano 8.7.1 heap.c
#include <stdio.h>
#include <stdlib.h>
int global = 10;
int bss_variable;
int main() {
    int local = 20;
    int *heap = malloc(sizeof(int));
    *heap = 30;
    printf("Code address : %p\n", (void *)main);
    printf("Data address : %p\n", (void *)&global);
    printf("BSS address : %p\n", (void *)&bss_variable);
    printf("Heap address : %p\n", (void *)heap);
    printf("Stack address : %p\n", (void *)&local);
    free(heap);
    return 0;
}
```

```
sadwika@DESKTOP-8CQA01L:~/Skill$ nano heap.c

sadwika@DESKTOP-8CQA01L:~/Skill$ gcc heap.c -o heap
sadwika@DESKTOP-8CQA01L:~/Skill$ ./heap
Code address : 0x5c883dbb21a9
Data address : 0x5c883dbb5010
BSS address : 0x5c883dbb5018
Heap address : 0x5c8875fca010
Stack address : 0x7ffd4dae9d8c
sadwika@DESKTOP-8CQA01L:~/Skill$ nano heap.c
sadwika@DESKTOP-8CQA01L:~/Skill$ |
```

3.Demand Paging



```
GNU nano 8.7.1 page.c
#include <stdio.h>
int main() {
    int page_table[4] = {1,-1,3,-1};
    int page;
    printf("Enter page number(0-3): ");
    scanf("%d", &page);
    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
        printf("Physical frame = %d\n",page_table[page]);
    }
    return 0;
}
```

```
sadwika@DESKTOP-8CQA01L:~/Skill$ nano page.c
sadwika@DESKTOP-8CQA01L:~/Skill$ gcc page.c -o page
sadwika@DESKTOP-8CQA01L:~/Skill$ ./page
Enter page number(0-3): 2
```

4.Virtual Physical Address

```
sadwika@DESKTOP-8CQA01L x + v
GNU nano 8.7.1 virtual.c *
#include <stdio.h>
int main() {
int page_table[] = {3,5,1,7};
int page_size = 100;
int virtual_address = 250;
int page = virtual_address/page_size;
int offset = virtual_address%page_size;
int frame = page_table[page];
int physical_address = frame*page_size+offset;
printf("Virtual Address = %d\n",virtual_address);
printf("Page Number = %d\n",page);
printf("Offset = %d\n",offset);
printf("Frame Number = %d\n",frame);
printf("Physical Address = %d\n",physical_address);
return 0;
}
```

```
sadwika@DESKTOP-8CQA01L:~/Skill$ nano virtual.c
```

```
sadwika@DESKTOP-8CQA01L:~/Skill$ gcc virtual.c -o virtual
sadwika@DESKTOP-8CQA01L:~/Skill$ ./virtual
Virtual Address = 250
Page Number = 2
Offset = 50
Frame Number = 1
Physical Address = 150
```

5.Memory Allocation

```
sadwika@DESKTOP-8CQA01L x + v
GNU nano 8.7.1 memory.c
#include <stdio.h>
#include <stdlib.h>
int main() {
int *p;
p = malloc(5*sizeof(int));
if(p==NULL) {
printf("Memory allocation failed\n");
return 1;
}
for(int i = 0; i<5; i++){
p[i]=(i+1)*10;
}
for(int i=0; i<5; i++) {
printf("%d",p[i]);
}
printf("/n");
free(p);
return 0;
}
```

```
sadwika@DESKTOP-8CQA01L:~/Skill$ nano memory.c
sadwika@DESKTOP-8CQA01L:~/Skill$ gcc memory.c -o memory
sadwika@DESKTOP-8CQA01L:~/Skill$ ./memory
1020304050/nsadwika@DESKTOP-8CQA01L:~/Skill$ |
```

6. Accessing Memory

```
sadwika@DESKTOP-8CQA01L x + v
GNU nano 8.7.1 accessingmemory.c
#include <stdio.h>
int main() {
    int *p = NULL;
    printf("Before accessing memory\n");
    *p = 100;
    printf("After accessing memory\n");
    return 0;
}

sadwika@DESKTOP-8CQA01L:~/Skill$ nano accessingmemory.c

sadwika@DESKTOP-8CQA01L:~/Skill$
sadwika@DESKTOP-8CQA01L:~/Skill$ gcc accessingmemory.c -o accessingmemory
sadwika@DESKTOP-8CQA01L:~/Skill$ ./accessingmemory
Before accessing memory
Segmentation fault ./accessingmemory
```